

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE CODE: ITA 0608

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15

Total Marks:25

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

*I **P.Sai Ganesh Reddy (192525111)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

P.Sai Ganesh Reddy

Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)

3. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

4. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling. (5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ANSWERS:

Question 1: Perceptron Model Fails to Converge

Possible Causes

- Learning rate is too high or too low.
- Incorrect weight initialization.
- Bias is missing or updated incorrectly.
- Error in the perceptron weight update rule.
- Training data may not be perfectly linearly separable.

Debugging Strategies

1. Verify that the dataset is linearly separable.
2. Check whether weights and bias are updated correctly.
3. Print weights after each iteration to detect errors.
4. Test different learning rates.
5. Validate the implementation using a small sample dataset.

Optimization Techniques

- Use a suitable learning rate (e.g., 0.01–0.1).
- Initialize weights with small random values.
- Normalize input features.
- Shuffle training data before each epoch.
- Stop training when convergence is achieved.

Output:

```
from sklearn.linear_model import Perceptron
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Generate dataset
X, y = make_classification(n_samples=200, n_features=2,
                          n_redundant=0, n_clusters_per_class=1,
                          random_state=42)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train Perceptron
model = Perceptron(max_iter=1000, eta0=0.1, random_state=42)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Weights:", model.coef_)
print("Bias:", model.intercept_)

... Accuracy: 0.85
Weights: [[ 0.39007782 -0.10906549]]
Bias: [-0.1]
```

Question 2: Multilayer Perceptron Shows Slow Convergence (5 Marks)

Possible Causes

- Vanishing gradients due to sigmoid activation.
- Poor weight initialization.
- Inappropriate learning rate.
- Local minima during optimization.
- Unnormalized input data.

Debugging Steps

1. Monitor training and validation loss.
2. Check gradient values during backpropagation.
3. Verify activation functions.
4. Test different learning rates.
5. Compare performance with different initializations.

Optimization Techniques

- Use ReLU activation instead of sigmoid.
- Apply Xavier or He weight initialization.
- Normalize input data.
- Use adaptive optimizers such as Adam.
- Implement learning rate scheduling and early stopping.

Output:

```

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train MLP
model = MLPClassifier(hidden_layer_sizes=(50,),
                      activation='relu',
                      solver='adam',
                      learning_rate_init=0.001,
                      max_iter=500,
                      random_state=42)

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

... Accuracy: 0.9666666666666667

```

Question 3: Decision Tree Overfitting (5 Marks)

Possible Causes

- Tree grows too deep.
- Noisy or irrelevant features.
- Small leaf nodes.
- Lack of pruning.

Debugging Steps

1. Compare training and testing accuracy.
2. Visualize the tree structure.
3. Check tree depth.
4. Identify noisy features.

5. Perform cross-validation.

Optimization Methods

- Apply pre-pruning by limiting tree depth.
- Use post-pruning techniques.
- Perform feature selection.
- Remove noisy data.
- Use cross-validation to choose optimal parameters.

Output:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Decision Tree with pruning
model = DecisionTreeClassifier(max_depth=3, random_state=42)

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

... Accuracy: 1.0
```

Question 4: SVM Underperforms on Nonlinear Dataset (5 Marks)

Possible Causes

- Incorrect kernel selection.
- Improper values of C and gamma.
- Features are not scaled.
- Presence of noisy data.

Debugging Techniques

1. Verify data preprocessing.
2. Test different kernels (RBF, Polynomial, Linear).
3. Scale features using normalization or standardization.
4. Analyze training and testing accuracy.
5. Perform cross-validation.

Optimization Methods

- Use the RBF kernel for nonlinear data.
- Tune C and gamma using Grid Search.
- Normalize all features.
- Remove irrelevant features.
- Apply feature engineering to improve classification.

Output:

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Feature Scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train SVM
model = SVC(kernel='rbf', C=1, gamma='scale')

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

... Accuracy: 1.0
```

